Manual técnico — Superfluid Brlt v2

Nómina en streaming sobre Besu 81234 · generado 2026-11-04

Tecnología (versión sencilla)

¿Qué es la plataforma?

En una frase

Una nómina que corre sobre un contrato: el admin deposita el salario y este se transmite Como una gota a gota; tú cobras cuando quieres.

Las piezas

- **BRLT**: el dinero (token) con 18 decimales.
- **BrltX**: la "versión en streaming" del dinero.
- **La app web** (`/admin` y `/suite`): las pantallas para administrador y empleado.
- **La red Besu** (chainId 81234): donde viven los contratos.

```
flowchart TB
    subgraph Navegador
        M[MetaMask firma todo]
    end
    subgraph AppWeb["App web (Cloud Run)"]
        P["Panel admin /admin"]
        S["Suite del Empleado /suite"]
    end
    subgraph Cadena["Besu 81234"]
        BRLT["BRLT (token)"]
        BRLTX["BrltX (streaming)"]
    end
    subgraph Datos["MongoDB"]
        D[Empleados · Flujos · Historial · Jobs]
    end
    M --> P -->|eth_call: lee estado| BRLTX
    M --> S -->|downgrade: cobro| BRLTX
    P --> D
    S --> D
```

⚠ *(nota del manual: el bloque "BR LTX" se dibujó por el archivo; es "BrltX"). Corregido en el gráfico* `arquitectura.svg` *de* `docs/imagenes/`.

¿Cómo sabe la app el estado real?

La regla de oro

El contrato manda. Cada vez que la app pinta una lista de flujos, vuelve a preguntarle al contrato por el número de flujo y actualiza su copia (MongoDB). Si alguien cambia algo a mano, la app se corrige La app se corrige sola en minutos.

> 📷 *Captura: 04-admin-resumen.png — url: /admin — panel admin con tarjetas de resumen*

¿Qué pasa cuando se pausa?

Pausa y reanudación

- Al pausar, el contrato **guarda la tasa** en un campo aparte.
- Al reanudar, el tiempo restante se **prolonga**: no pierdes devengo.
- Todo esto lo hace el servidor leyendo `getFlow` y `getPending`.

```
sequenceDiagram
    participant A as Admin (MetaMask)
    participant X as BrltX
    participant E as Empleado
    A->>X: wrapAndCreateFlow(10 BRLT, 6 días)
    X-->>A: FlowCreated #N (tasa floor)
    A->>X: updateFlow(#N, 0) — pausa
    X->>E: liquida el tramo (minta BrltX)
    A->>X: updateFlow(#N, tasa) — reanudar
    A->>X: deleteFlow(#N) — terminar
    X->>E: paga lo devengado · remanente vuelve al admin
```

Seguridad en palabras simples

¿Quién puede hacer qué?

- El admin necesita estar **autorizado** (allowlist) e iniciar sesión firmando un mensaje (SIWE).
- Cada empleado solo accede **a su propia Suite**: nadie puede pedir tus flujos.
- Cada acción peligrosa pide otro paso (CSRF + firma).
- El **cobro lo firma tu wallet**: la web solo dice que lo hiciste si el contrato lo confirma.

Dependencias (versión sencilla)

¿Qué programa necesita mi ordenador?

Para usar la plataforma (empleado o admin)

Solo un navegador (Chrome/Edge/Firefox) y **MetaMask** instalado en el navegador.

Para trabajar en el código

- **Node.js 20+** (probado con 24).
- **Foundry** para los contratos (`forge` , `cast`).
- MongoDB **no hace falta instalarla**: el proyecto puede usar una "Mongo de bolsillo" que se borra al cerrar (útil en Windows).

Programas que usa el proyecto

Del lado del servidor

Programa	Qué hace
Next.js 15	El "motor" de la web.
ethers 5	El traductor con MetaMask y la red.
MongoDB driver	Guarda las fichas, flujos e historial.
SIWE lib	Comprueba el "iniciando sesión con la wallet" (firma).
Zod	Vigila que los datos que llegan tengan la forma correcta.

Del lado del desarrollador

Programa	Qué hace
Vitest	Ejecuta las pruebas rápidas.
Playwright	Controla un Chrome "invisible" para pruebas y pantallazos.
tsx	Ejecuta los utilitarios sin compilar.
TailwindCSS	Los estilos visuales.

Cómo revisar que todo está en orden

Comandos de control

```
cd web
npm test              # pruebas rápidas (43)
npm run smoke:h4      # prueba completa contra la red de práctica
npm run e2e:ui        # prueba con el navegador automático
```

Si todos acaban con “**TODO OK**”, el sistema está sano.

Implementación (versión sencilla)

El ciclo de vida de un salario

De principio a fin

1. **El admin crea el flujo**: aprueba BRLT, lo deposita en el contrato y se abre un "grifo" hacia tu wallet.
2. **Cada segundo cuenta**: el grifo reparte el importe entre los días del periodo (6, 15, 30 o 90 días).
3. **Tú decides cuándo cobrar**: pulsas «Cobrar» y tu wallet recibe BRLT real.
4. **Si el flujo termina** (vence o el admin lo cierra): lo ganado pasa a tu saldo y lo que aún no corresponde vuelve al admin. Nada se pierde.

```
timeline
    title Nómina de 6 días (10 BRLT)
    Día 0 : Flujo creado (depositado 10 BRLT)
    Día 1 : ~1,67 BRLT devengado (acumulando)
    Día 3 : Pausa → liquida lo devengado → tasas congeladas
    Día 3 : Reanudar → el "fin del flujo" se aplaza
    Día 6 : Flujo vencido → pendiente de liquidar
    Día 6 : Liquidar (público) → acumulado → BrltX cobrable
    Siempre : «Cobrar» → BRLT a tu wallet
```

¿Y si cambio de wallet?

Cambio de receptor seguro

- Lo pides en el panel, firmas con la **wallet nueva** y el admin firma la confirmación.
- El contrato cierra el flujo viejo y abre uno **nuevo** hacia tu wallet nueva con el saldo restante. **Sin pagar dos veces.**

> 📷 *Captura: 06-admin-empleados.png — url: /admin/empleados — cambio de wallet con doble firma*

¿Qué pasa si la web se apaga?

Se acabó la sesión

- Si la web cae, **los flujos siguen en la cadena** (no dependen de la web).
- Cuando la web vuelve, los "trabajos automáticos" (jobs) vuelven a leer la cadena y **corrigen su copia** en la base de datos.

> 📷 *Captura: 08-admin-jobs.png — url: /admin/jobs — panel de jobs con estado y botón Ejecutar*

La regla del cobro (lo importante)

Protección del empleado

- El saldo que ves como **“Cobrable ahora”** es dinero ya tuyo: lo puedes cobrar aunque el contrato esté en pausa de mantenimiento.
- La web **registra tu cobro solo si la cadena lo confirma**; un comprobante falso se rechaza (“tx_no_verificada”).

Despliegue (versión sencilla)

¿Dónde vive la aplicación?

Hoy (red de práctica)

- **Contratos**: desplegados en la red de práctica Besu (chainId 81234), bloque 6.412.104. Nada que instalar: la app ya está conectada.
- **App**: se puede correr en cualquier ordenador con Node 20+.

Cómo poner la aplicación en marcha

Modo rápido (5 comandos)

```
cd web
npm install
cp .env.example .env.local    # y rellena lo que pide
npm run seed                  # prepara la base y tu wallet admin
npm run dev                   # http://localhost:3000
```

Modo container (para servidor)

```
npm run build
npm run verify:standalone     # prueba el "ejecutable" de producción
gcp/deploy.sh                 # sube a Google Cloud Run (requiere acceso)
```

Panel admin con tarjetas de resumen *(El panel admin tal y como lo verá el administrador tras iniciar sesión.)*

¿Qué pide la app para conectarse?

Variables importantes (solo el administrador técnico)

- `MONGODB_URI` / `MONGODB_DB` — dónde vive la base de datos.
- `SIWE_DOMAIN` — **debe coincidir exactamente** con la dirección web usada.
- `NEXT_PUBLIC_BRLT_ADDRESS` / `NEXT_PUBLIC_BRLTX_ADDRESS` — las direcciones de los contratos.
- `ADMIN_BOOTSTRAP_WALLET` — la wallet que puede entrar como admin.

Healthcheck

¿Está vivo el sistema?

Una visita a `/api/health` responde:

- `200 ok` — base de datos y red funcionando.

- `503 degradado` — algo no responde (la app no está muerta solo degradada).

Diccionario de datos (versión sencilla)

¿Dónde vive cada cosa?

Dos manos que trabajan juntas

1. **La cadena (Besu)** guarda lo que cuesta dinero: quién cobró, cuánto y cuándo.
2. **La base MongoDB** guarda papeleo: fichas, proyectos, permisos, historial y avisos.
3. La app **compara siempre** las dos y se corrige si difieren.

Cosas que la base guarda

Pequeño glosario

Cosa	Qué es
Empleado	Cada persona con su wallet.
Proyecto	La “familia” a la que pertenece un empleado (info).
Flujo	El “grifo” del salario con su número (#1, #2…).
Programado	Una nómina “apuntada” para lanzarse más adelante.
Lote	Hasta 10 flujos creados de golpe, con control anti-duplicados.
Historial (`flows_history`)	El diario de los eventos del contrato.
Trabajo (`job_runs`)	Las veces que la app se “auto-revisa”.
Auditoría	Quién hizo qué y cuándo (sin secretos ni datos íntimos).

Datos personales (y tus derechos)

Reglas del juego

- Email y teléfono **se borran al instante** si lo pides («Borrar mis datos»).
- A los **30 días**, la app anonimiza el nombre automáticamente (ya no se sabe quién es).
- Tu **wallet** se aguanta hasta **90 días** después de la baja para que puedas retirar lo que te debían; después la compte queda sellada.

```
mindmap
  root((Datos))
    Cadena
      Flujos y pagos
      Eventos (historial)
      Escrow y tasas
    MongoDB
      Empleados y proyectos
      Sesiones y permisos
      Auditoría y jobs
    Derechos
      Borrado inmediato
      Anonimizado a 30 días
      Retiro hasta 90 días
```

Estados de un flujo

Los cuatro estados

```
stateDiagram-v2
    [*] --> activo: se crea el flujo
    activo --> pausado: pausa del admin
    pausado --> activo: reanudación
    activo --> vencido: pasa su tiempo
    vencido --> terminado: liquidación (cualquiera)
    activo --> terminado: terminación del admin
    terminado --> [*]
```

Diagrama relacional (versión sencilla)

El mapa de la base de datos

Relaciones que importan

- Un **Proyecto** agrupa a varios **Empleados** (opcional).
- Cada **Empleado** puede tener varios **Flujos** (normalmente uno activo por periodo).
- Un **Flujo** puede venir de un **Lote** (por si se crearon varios a la vez).
- Cada **Flujo** deja rastro en el **Historial** al igual que su despedida.
- Los **Trabajos (jobs)** anotan las veces que la app se revisa a sí misma.

```
flowchart LR
    PRO[Proyectos] -->|agrupa| EMP[Empleados]
    EMP -->|tiene| FLO[Flujos]
    EMP -->|tendrá| SCH[Programados]
    BAT[Lotes] -->|crea| FLO
    FLO -->|deja rastro| HIS[Historial]
    EMP --> AUD[Auditoría]
    JOBS[Trabajos] --- NOTA((auto-revisión))
    style NOTA fill:#f1f5f9,stroke:#94a3b8
```

La doble contabilidad

Cadena + base (y nunca al revés)

- **Cadena** = dinero y verdad (no se puede borrar).
- **Base** = conveniencia (se puede reconstruir).
- Si alguien borra algo de la base, la app lo re-lee del **bloque inicial de despliegue** y lo repara. Ese es el papel de los jobs del panel `/admin/jobs`.